

COOKING STAR

완벽 방어! 백엔드 기술 면접 해설서

총 14개의 핵심 실전 코드 분석 및 방어 시나리오

MyBatis 동적 SQL 에러 방어

```
// MyRecipeController.java
@PostMapping
("recentList")
public
List getRecentList( ... ) {
    if (recipeNums = null || recipeNums.isEmpty()) {
        return new ArrayList<>();
    }
    return
    myRecipeService.fileList(recipeNums); }
```

Q 조회 결과가 없을 때 null 대신 빈 리스트를 반환한 이유는?

- 💡 **서버 500 에러 방어:** null 데이터가 MyBatis 로 넘어가면 WHERE IN ()이라는 구문 오류가 발생합니다.
- 💡 **프론트엔드 보호 (Early Return):** JS가 null을 받으면 forEach 루프에서 화면이 멈추지만, [] (빈 배열)을 주면 에러 없이 안전하게 렌더링을 마칩니다.

다중 첨부파일 (1:N) 수정 상태 검증

```
// MyCookingService.java update
int
maxFileCount = 5;
// ... existingFileCount, deleteCount, newCount 연산
// 최종 이미지 갯수 연산 수식
int finalCount = existing - delete + newCount;
if
(finalCount > maxFileCount) {
throw new
IllegalArgumentException(
"최대 5장"
); }
```

Q 프론트엔드(JS)에서 개수를 막았는데 백엔드에 또 수식을 넣은 이유는?

- 💡 API 직접 호출(바이패스) 방어:** JS의 경고창은 Postman 등을 통한 악의적인 접근을 막지 못해 서버 스토리지가 터질 수 있습니다.
- 💡 상태 변이 통제:** '기존 삭제'와 '신규 추가'가 얹힌 복잡한 수정 로직에서, 철저한 **수학적 수식 검증**을 통해 DB-파일 상태의 무결성을 지켜냈습니다.

CASCADE 연쇄 삭제와 고아 파일 방지

```
// MyRecipeService.java delete
// DB 폭파 전 명단 백업
List targetFile = detail.getFileDTO();
// DB 삭제 (자식 데이터 연쇄 삭제)
int
result = myRecipeMapper.delete(DTO);
if (result > 0 && targetFile != null) {
for
(RecipeFileDTO file : targetFile) {
fileManager.fileDelete(
"myrecipe"
, file); }
}
```

Q 굳이 변수에 파일 정보를 '백업'해두고 나중에 지우는 이유는?

- 💡 **고아 파일 방지:** DB 레시피를 삭제하면 CASCADE로 파일 테이블 기록도 날아갑니다. 백업을 안 해두면 **하드디스크에서 무슨 이름의 파일을 지워야 하는지 영영 알 수 없습니다.**
- 💡 **안전한 삭제 순서:** 무조건 DB 처리를 먼저 성공시킨 후 백업 명단으로 실제 파일을 지워 데이터 유실(엑스박스)을 막았습니다.

결제 완료와 RedirectAttributes (PRG 패턴)

```
// BuyListController.java orderComplete
public
String order( ... ,
    RedirectAttributes allot
) {
    int
    result = buyListService.setBuyList( ... );
    if
    (result > 0) {
        allot.addFlashAttribute("msg", "결제 완료");
        return "redirect:/buylist/list";
    }
}
```

 **결제 후 일반 Model(포워드)을 쓰지 않고 리다이렉트 처리한 이유는?**

 **중복 결제 방지:** 포워드로 넘어가면 새로고침(F5) 시 이전의 '결제 요청'이 또 실행됩니다. 이를 강제로 끊어내는 **PRG 패턴**을 적용했습니다.

 **Flash Attribute 활용:** 리다이렉트를 하면 Model이 소멸하므로, 세션에 1회성으로 메시지를 숨겼다 보여주고 자동 삭제하는 방식을 택했습니다.

Toss Payments S2S 검증 로직

```
// 자바스크립트 성공 → 백엔드로 데이터 전송  
// Backend (Toss API 서버로 2차 검증 요청)
```

```
JSONObject obj = new JSONObject();
```

```
obj.put("orderId", orderId);
```

```
obj.put("amount", amount);
```

```
obj.put("paymentKey", paymentKey);
```

```
// 토스 서버에 최종 Confirm 요청
```

```
HttpEntity request =
```

```
new
```

```
HttpEntity<>( ... );
```

Q JS에서 이미 결제 성공이 났는데 백엔드에서 다시 검증하는 이유는?

 **클라이언트 위변조 차단:** 해커가 브라우저 개발자 도구에서 결제 금액을 '500원'으로 조작해서 서버로 쓸 수 있습니다.

 **Server-to-Server (S2S):** 프론트의 성공은 '가승인'입니다. 우리 백엔드가 토스 서버에 직접 물어봐서 **DB의 물건값과 실제 결제 금액이 일치하는지 교차 검증**해야 합니다.

BCrypt 알고리즘 도입 배경

```
// SecurityConfig.java
```

```
@Bean
```

```
public PasswordEncoder passwordEncoder() {
```

```
    return new BCryptPasswordEncoder();
```

```
}
```

```
// 회원가입 로직 내부
```

```
memberDTO.setPw(passwordEncoder.encode(pw));
```

Q 일반 해시(SHA-256)를 쓰지 않고 BCrypt를 선택한 이유는?

💡 솔팅(Salting) 기본 내장: 똑같은 '1234'를 입력해도 랜덤한 소금(Salt)을 쳐서 암호화하므로, 해커가 레인보우 테이블로 비밀번호를 역추적할 수 없습니다.

💡 키 스트레칭: 연산 속도를 일부러 늦춰서 무차별 대입(Brute Force) 공격 시 엄청난 시간이 걸리도록 서버 보안을 강화했습니다.

비동기 세션 만료 제어 (Early Return)

```
// CartController.java delete
@PostMapping
(
    "delete"
)
@ResponseBody
public int
delete(CartDTO cartDTO, Principal principal) {
    // 1. 로그인 안 되어 있으면 거절 신호 송출
    if (principal == null) return -1;
    return
    cartService.delete(cartDTO); }
```

Q 실패면 보통 0을 주는데, 굳이 '-1'을 리턴한 이유와 종괄호를 생략한 이유는?

- 💡 프론트엔드와의 약속:** 단순 실패(0)와 '세션 만료(-1)'를 명확히 구분하여, JS가 -1을 받으면 즉시 로그인 페이지로 튕겨내도록 설계한 플래그(Flag)입니다.
- 💡 가드 클로즈(Guard Clause):** 코드가 한 줄일 때 종괄호를 생략하는 실무형 '얼리 리턴' 패턴으로 로직을 깔끔하게 정리했습니다.

Set을 활용한 대량 API 데이터 대조

```
// SpotService.java search
List mySpotList = spotMapper.myList(username);
// 내 맛집 URL을 Set으로 변환
Set savedUrlSet = new HashSet<>();
for
(SpotDTO mySpot : mySpotList) {
    savedUrlSet.add(mySpot.getPlaceUrl()); }
// O(1) 성능으로 존재 여부 비교
if
(savedUrlSet.contains(spotDTO.getPlaceUrl())) { ... }
```

Q 내 리스트(List)를 그대로 비교하지 않고 굳이 HashSet으로 옮겨 담은 이유는?

- 💡 시간 복잡도 최적화:** List.contains는 끝까지 뒤져야 해서 **O(N)**이 걸리지만, HashSet은 해시 알고리즘으로 **O(1)**만에 데이터를 찾아냅니다.
- 💡** 카카오 API 결과 15개와 내 DB 전체를 이중 루프로 대조할 때 발생할 수 있는 서버 병목 현상을 자료구조 변환으로 방어했습니다.

SQL 레벨의 정규식 문자열 정제

```
SELECT
  TRIM(REGEXP_REPLACE(
    "LOG_TITLE",
    '레시피 추천|만들기|방법', '', 'g'
  )) AS "keyword",
COUNT(*) AS
"count"
FROM
"LOG"
GROUP BY
"keyword"
ORDER BY
"count"
DESC LIMIT 5
```

Q Java에서 문자열을 자르지 않고 DB 쿼리에서 직접 정규식을 돌린 이유는?

- 💡 **서버 메모리 보호:** 수천 개의 로그를 Java 메모리에 다 올려서 자르고 집계하면 애플리케이션 서버 부하가 엄청납니다.
- 💡 **집계 효율성 극대화:** DB 엔진 단에서 정규식으로 불필요한 단어를 지운 순수 '키워드'를 즉시 GROUP BY로 묶어, 가벼운 Top 5 데이터만 서버로 넘기게 최적화했습니다.

ON CONFLICT (Upsert) 쿼리 활용

```
INSERT INTO ai_recommend (menu_name, time_zone) VALUES  
(#{menuName}, #{timeZone})
```

```
ON CONFLICT (time_zone)
```

```
DO UPDATE SET
```

```
menu_name = EXCLUDED.menu_name;
```

Q 이 쿼리의 장점은 무엇이며, 프로젝트 내 어디에 확장 적용하면 좋을까요?



통신 1회 단축 (원자성): 자바에서 조회(Select) 후 분기하여 Insert/Update를 하면 쿼리를 2번 날려야 하지만, 이 구문은 DB 단에서 1번의 쿼리로 안전하게 '있으면 수정, 없으면 삽입'을 처리합니다.



확장: 장바구니에 똑같은 상품을 담을 때 '수량만 누적'시키는 로직에 적용하면 완벽합니다.

MyBatis 반환값(int)의 누적 처리

```
// BuyListService.java
int
result = 0;
for
(BuyListDTO buy : list) {
    // MyBatis Insert 결과를 누적
    result += buyListMapper.setBuyList(buy);
}
return
result;
// 최종 성공 개수 반환
```

Q INSERT 쿼리인데 도대체 어떤 값이 나오길래 += 연산자로 더할 수 있나요?

-
- 💡 행(Row)의 개수 반환:** MyBatis에서 DML(Insert, Update, Delete) 쿼리가 성공적으로 실행되면, 해당 쿼리의 영향을 받은 '행의 개수(int)'를 반환합니다.
- 💡** 1건이 인서트 되면 1이 반환되므로, 장바구니 리스트를 돌면서 +=로 더하면 최종적으로 **"성공적으로 결제된 상품의 종류 수"**를 정확히 계산해 뷰에 전달할 수 있습니다.

프로퍼티 주입과 민감 정보 격리

```
// SearchService.java
```

```
@Service
```

```
public class
```

```
SearchService {
```

```
    @Value("${naver.api.client.id}")
```

```
    private String clientId;
```

```
    @Value("${naver.api.client.secret}")
```

```
    private String clientSecret;
```

```
// ... HTTP 헤더에 변수 사용
```

```
}
```

 **API Key를 코드에 직접 적지 않고 @Value로 빼서 주입받은 이유는 뭔가요?**

 **보안 사고 원천 차단:** 시크릿 키를 자바 코드에 하드코딩해서 GitHub에 올리면 요금 폭탄 등 치명적인 보안 사고가 납니다.

 **환경 분리 유지보수:** 민감 정보를 application.properties로 분리하고 gitignore에 등록하여, 코드를 다시 빌드할 필요 없이 환경 파일만 바꿔 끼우도록 유연성을 높였습니다.

@ControllerAdvice로 전역 데이터 주입

```
// GlobalAdvice.java
```

```
@ControllerAdvice
public class
GlobalAdvice {
    @ModelAttribute("todayVisit")
    public int
    getTodayVisit() {
        return
        visitService.getToday(); } }
```

Q 통계 데이터를 개별 컨트롤러에서 넘기지 않고
별도 Advice 클래스로 뺀 이유는?

-  **보일러플레이트(중복) 제거:** 방문자 수는 사이트 하단 (Footer) 등 모든 화면에서 보여줘야 합니다.
-  수십 개의 컨트롤러에 일일이 model.addAttribute 코드를 넣는 대신, **전역(Global) 설정**을 통해 어떤 URL로 접속하든 자동으로 뷰에 데이터가 꽂히도록 설계했습니다.

@PostConstruct 타임존 강제 동기화

```
// CookingStarApplication.java
@SpringBootApplication
public class
CookingStarApplication {
    @PostConstruct
    public void started() {
        TimeZone.setDefault(
            TimeZone.getTimeZone("Asia/Seoul")
        );
    }
}
```

Q 서버 기동 시점에 타임존을 강제로 'Asia/Seoul'로 고정한 이유는?

- 💡 **클라우드 배포 인프라 대응:** 로컬 PC와 달리 AWS 같은 클라우드 서버는 기본 시간이 UTC(표준시)로 되어 있어 한국과 9시간의 오차가 납니다.
- 💡 어떤 인프라(OS, 서버 위치)에 배포되더라도 **자바 애플리케이션의 시간대가 무조건 한국 기준으로 동작하도록 강제**하여, 등록일/결제일 오류를 막았습니다.

 실전! 내 코드에 없는 방어 로직 공격이 들어왔을 때

"모른다" 대신 "성장할 준비가 됐다"고 대답하세요.

[1단계 인정]: "네, 솔직히 말씀드리면 현재 제 코드에는 (동시성 제어/확장자 검사) 방어 로직이 구현되어 있지 않습니다."

[2단계 공감]: "면접관님 지적대로 그 처리가 없다면, 따닥 버튼 클릭 시 (갱신 손실 / 보안 사고) 같은 치명적인 문제가 발생할 수 있음을 인지했습니다."

[3단계 방향]: "지금 보완한다면 실무에서 많이 쓴다는 (Redis Lock / Apache Tika) 같은 기술을 도입해 보고 싶은데, 입사 후 꼭 적용해 보겠습니다."